

Minerva: una Pipeline di Parsing e Valutazione Automatica per Sorgenti Web Eterogenee

Laura De Michelis, Matricola 2119887

Andrea Giuliotti, Matricola 2107814

Alessandro Di Nitto, Matricola 2109155

Sapienza Università di Roma

1 Obiettivo del Progetto

Con questo progetto abbiamo costruito una pipeline completa che scarica pagine da quattro domini web diversi (Wikipedia in italiano, AppleVis, Basketball-Reference e OndaRock), ne estrae il testo utile, lo confronta con un Gold Standard scritto da noi e lo fa valutare anche da un LLM locale. Tutto il sistema è esposto tramite API REST e una Web UI, e gira in quattro container Docker (backend, frontend, database, LLM).

2 Organizzazione del Codice

Il backend (FastAPI, porta 8003) sta in `backend/src/` ed è diviso in quattro parti. In `parsers/` c'è una classe astratta `BaseDomainParser` che si occupa di tutto quello che è comune a ogni sito (aprire il browser di Crawl4AI, scaricare l'HTML, riparsarlo da una stringa già salvata), mentre ogni dominio ha la sua sottoclasse con il selettore CSS giusto, le esclusioni per togliere il rumore e la pulizia finale del Markdown – abbiamo capito quali selettori usare guardando a mano l'HTML di ogni sito. Un `ParserFactory` sceglie il parser giusto in base al dominio dell'URL. In `evaluation/` c'è tutto quello che serve a ripulire il Markdown prima del confronto, calcolare le metriche e parlare con Ollama per il giudizio dell'LLM. In `db/` c'è la connessione a MariaDB, lo schema e le funzioni per leggere/scrivere, incluso lo script che popola il database all'avvio. In `models/` ci sono gli schemi Pydantic di input/output di ogni endpoint. Il frontend (FastAPI + Jinja2, porta 8004) ha quattro pagine (home, parser & evaluation, gold standard builder, stats) e parla *solo* con le API REST del backend, mai direttamente col database.

3 Valutazione Parser

Come richiesto abbiamo usato `token_level_eval` (precision, recall, F1 sui token separati da spazio, dopo aver tolto il Markdown da entrambi i testi), e in più abbiamo aggiunto ROUGE-1: a differenza della prima metrica, che guarda solo se una parola c'è o no, ROUGE-1 conta anche quante volte compare, quindi si accorge anche se il parser ripete o salta pezzi di testo.

Gli F1 medi sulle 10 pagine di ogni dominio, letti da `GET /db_stats` (pre-calcolati a ogni avvio, vedi Sezione 5), sono: `it.wikipedia.org` 0.90, `www.applevis.com` 0.97, `www.basketball-reference.com` 0.77, `www.ondarock.it` 0.97. Per `basketball-reference.com` lo scope dipende dal tipo di pagina: sulle schede giocatore/squadra/campionato copriamo solo il blocco anagrafico (`#meta`, `#bling`, `.stats_pullout`), non le tabelle partita-per-partita; sulle pagine `/playoffs/` invece il Gold Standard ufficiale del corso include anche le tabelle di sintesi Team/Opponent e i leader statistici – nascosti nell'HTML dentro commenti `<!-- -->` rivelati via JavaScript in un browser reale, quindi “scommentati” esplicitamente solo su quelle pagine prima del parsing.

Durante la costruzione del Gold Standard, e poi confrontando i risultati con lo script di valutazione ufficiale del corso, abbiamo trovato alcuni problemi. Il parser HTML della libreria standard di Python si fermava senza avvisi a metà pagina su markup irregolare, tagliando il 95% del contenuto reale – risolto con un parser più permissivo. Piccoli residui di Markdown (spazi in più, parole attaccate a un grassetto dopo aver tolto i simboli) fanno perdere qualche punto di F1, limite della tokenizzazione a spazi più che del parser. Infine, sullo script ufficiale il Gold

Standard di `www.ondarock.it` ha caratteri accentati mancanti già nell'`html_text` di partenza (persi a monte, non nel nostro parsing) mentre il `gold_text` li ha: un limite del fixture non recuperabile, che tiene l'F1 di quella pagina a 0.72 pur sopra soglia.

4 Valutazione dei Judge

Come LLM Judge abbiamo scelto `llama3.2:3b` (Meta, circa 2GB di RAM) tra i cinque modelli proposti, principalmente perché è il più veloce su CPU e la macchina su cui abbiamo sviluppato non è potentissima: una singola valutazione ci mette da circa 1 a circa 4 minuti, a seconda che il modello sia già caricato in RAM o debba essere ricaricato da disco. Il prompt chiede una risposta in JSON con `score` (da 1 a 5) e `feedback`, con temperatura bassa (0.1) per avere un giudizio il più possibile coerente a parità di input. Per il parsing della risposta abbiamo previsto quattro livelli di fallback in cascata (JSON diretto, poi il primo blocco `{...}` che si trova nel testo, poi i singoli campi via regex, infine una cifra isolata), e solo se proprio niente funziona ricadiamo su un punteggio neutro.

Una cosa che ci ha colpito: su tutte e 40 le pagine del Gold Standard il punteggio è quasi sempre 4/5, indipendentemente dal vero F1 (0.85–0.99). Per escludere un bug abbiamo dato al judge un testo scollegato dal Gold Standard: il punteggio è sceso a 2/5, con feedback che citava l'argomento sbagliato – il modello legge davvero il contenuto, ma sembra riservare il 5/5 quasi solo a corrispondenze testuali identiche, trattando ogni imperfezione minima allo stesso modo. Comportamento noto dei modelli piccoli usati come giudici a bassa temperatura, non un difetto nostro.

Per curiosità abbiamo provato anche gli altri modelli proposti: `qwen3:4b` (incompatibile col nostro codice, mette il ragionamento in un campo `thinking` separato da `response`), `phi4-mini` (punteggi peggiori e meno informativi), `gemma3:e2b` (7GB, troppo pesante: saturava la RAM e mandava in timeout il browser di scraping) e `ministral-3:3b` (generazione a 1 token/s sulla nostra CPU: una valutazione oltre 7 minuti, poi crash del processo Ollama). Confermano che il collo di bottiglia è l'hardware di sviluppo (CPU senza GPU, RAM condivisa tra Docker/Ollama/browser) più che il modello, e che `llama3.2:3b` resta il compromesso più stabile.

Abbiamo anche provato a dare al judge una

rubrica esplicita (criteri diversi per ogni punteggio 1–5) su tre pagine con F1 molto diverso (1.00, 0.92, 0.81): punteggio 4/5 in tutti e tre i casi, col prompt vecchio e col nuovo. Il feedback testuale cambiava correttamente (il modello legge i testi), il punteggio no: un limite di calibrazione del modello, non qualcosa che abbiamo voluto “correggere” a mano per non falsare il giudizio dell'LLM.

5 Organizzazione del DB

Nel database MariaDB abbiamo le due tabelle obbligatorie `web_resources` e `gold_standard`, collegate uno-a-uno con una chiave esterna su `url` (con cascade in eliminazione), più due tabelle nostre: `evaluations` (i risultati di `token_level_eval` e ROUGE-1 per ogni pagina) e `llm_judgments` (punteggio, feedback e nome del modello usato per il giudizio). Tutte le query passano da un livello di repository e sono parametrizzate. All'avvio, uno script di seed legge `gs_data/*.json` e riempie `web_resources/gold_standard` con un `upsert`, così l'HTML grezzo resta sempre disponibile per rifare il parsing senza ricaricare niente. Subito dopo pre-calcola anche `evaluations` (metriche token-level/ROUGE-1 su ogni pagina, economiche) e `llm_judgments` (un giudizio LLM per la prima pagina di ogni dominio, per non allungare troppo l'avvio): così `GET /db_stats` ha sempre dati pronti anche prima che un utente chiami `/evaluate` o `/full_gs_eval` (che aggiornano le stesse tabelle quando usati).